

Kear ♥

Smart Medicine Reminder & Care Coordination Web Application

Academic Mini Project — Software Requirements & Design Document

Technology Stack: PHP 8.x | MySQL | HTML/CSS/JS

Version 1.0 | 26 June 2026

1. Introduction	4
1.1 Project Overview	4
1.2 Problem Statement	4
1.3 Project Objectives	4
1.4 Scope	4
1.5 Definitions and Acronyms	5
2. Stakeholders & User Roles	6
2.1 Overview of Roles	6
2.2 Patient	6
2.3 Caretaker	6
2.4 Doctor	6
2.5 Administrator	7
3. Functional Requirements	8
3.1 User Authentication & Registration	8
3.2 Medicine Schedule Management	8
3.3 Reminder System	8
3.4 Prescription Upload & Parsing	9
3.5 Doctor Connection	9
3.6 Doctor–Caretaker Chat	9
3.7 Dose Logging & History	9
3.8 Admin Module	10
4. Non-Functional Requirements	11
4.1 Security	11
4.2 Performance	11
4.3 Usability	11
4.4 Reliability	11
4.5 Maintainability	12
5. Database Design	13
5.1 Overview	13
5.2 Table Definitions	13
5.2.1 users	13
5.2.2 doctor_profiles	13
5.2.3 patient_caretaker	14
5.2.4 patient_doctor	14
5.2.5 medicines	14
5.2.6 schedules	15
5.2.7 dose_logs	15
5.2.8 prescriptions	16
5.2.9 messages	16
5.2.10 notification_logs	16
5.2.11 support_tickets	17
5.2.12 ticket_replies	17
5.2.13 admin_activity_log	18
5.2.14 cron_health	18
6. Module Descriptions	19
6.1 Authentication Module	19
6.2 Schedule Manager Module	19
6.3 Reminder Engine (Cron Module)	19
6.4 Prescription Parser Module	20
6.5 Doctor Connection Module	21

6.6 Chat Module.....	21
6.7 Admin Module	22
6.7.1 Admin Dashboard	22
6.7.2 User Management.....	22
6.7.3 Doctor Verification.....	22
6.7.4 Prescription Audit.....	22
6.7.5 Missed Dose Reports.....	22
6.7.6 Support Ticket System	23
6.7.7 Chat Moderation.....	23
6.7.8 System Health.....	23
7. Application Page Structure	24
7.1 Public Pages.....	24
7.2 Patient Pages.....	24
7.3 Caretaker Pages	24
7.4 Doctor Pages	25
7.5 Admin Pages.....	25
8. Project Folder Structure	26
8.1 Configuration File	26
9. External Libraries & Dependencies	28
9.1 PHP Libraries (Manual / Pre-bundled — No Composer Required)	28
9.2 Third-Party API Services	28
9.3 Front-End Libraries (CDN)	28
10. Security Considerations	30
10.1 Input Validation & SQL Injection Prevention	30
10.2 CSRF Protection	30
10.3 File Upload Security	30
10.4 Admin Security	31
11. Implementation Plan	32
11.1 Development Phases	32
11.2 Recommended Development Environment	32
11.3 Testing Checklist.....	32
12. Future Enhancements	34
13. Appendix	35
13.1 Database Schema (SQL)	35
13.2 Library Setup Reference	35
13.3 References.....	36

1. Introduction

1.1 Project Overview

MediRemind is a PHP and MySQL-based web application designed to assist patients in managing their medicine schedules and ensure timely medication intake. The system bridges patients, caretakers, doctors, and administrators into a single coordinated platform. It automates medicine reminders via email and SMS, tracks missed doses, parses prescription documents, facilitates secure doctor–caretaker communication, and provides administrative oversight.

This document serves as the complete Software Requirements and Design Specification for the MediRemind project, covering system purpose, stakeholders, functional and non-functional requirements, database design, module descriptions, and implementation guidance.

1.2 Problem Statement

Non-adherence to prescribed medication schedules is one of the most common and preventable causes of disease progression, hospital readmission, and treatment failure. Patients — especially the elderly, chronically ill, or those managing multiple medicines — frequently forget doses, take incorrect quantities, or have no immediate way to alert a caretaker or doctor when a dose is missed.

Existing reminder apps are either too generic, lack doctor integration, or do not support caretaker involvement. MediRemind solves this by combining automated reminders, prescription management, real-time communication, and administrative supervision in a single web platform accessible from any device.

1.3 Project Objectives

- Allow patients to create and manage their own medicine schedules with time-based reminders.
- Automatically notify patients and caretakers via email and SMS when a medicine dose is approaching.
- Alert caretakers immediately when a patient misses a scheduled dose.
- Allow caretakers and verified doctors to update or modify patient medicine schedules.
- Enable prescription uploads in PDF or DOCX format with automatic text extraction and schedule pre-filling.
- Provide a real-time chat interface between caretakers and doctors for medical coordination.
- Allow patients to connect with doctors directly, receive doctor-prescribed schedules, and disconnect after recovery.
- Provide an admin module for user supervision, doctor verification, support tickets, and system health monitoring.

1.4 Scope

The scope of this project is limited to a web-based application built using PHP 8.x and MySQL. The system is intended for use in a local or hosted server environment. It does not include a mobile application, but the web interface will be responsive for mobile browsers. No Composer or package manager is used; all PHP libraries are included as pre-bundled files with their own autoloaders. Front-end dependencies are loaded via CDN. Integration with third-party services is limited to PHPMailer for email, Fast2SMS or Twilio for SMS, smalot/pdfparser (pre-bundled build) for PDF extraction, and PHPOffice/PhpWord (pre-bundled build) for DOCX extraction.

1.5 Definitions and Acronyms

Term / Acronym	Definition
Patient	The end user who takes medicine and whose schedule is managed by the system.
Caretaker	A trusted person (family member, nurse) who monitors the patient's adherence.
Doctor	A verified medical professional who prescribes and manages medicine schedules.
Admin	System administrator responsible for oversight, verification, and support.
CRUD	Create, Read, Update, Delete — basic database operations.
Cron Job	A time-based Linux scheduler that runs PHP scripts automatically at intervals.
PHPMailer	An open-source PHP library used for sending emails via SMTP.
SMS	Short Message Service — text message notification to a mobile number.
PDF	Portable Document Format — used for uploading prescription files.
DOCX	Microsoft Word document format — alternative for prescription uploads.
CSRF	Cross-Site Request Forgery — a web security vulnerability that must be prevented.
DXA	Device-independent unit used in document formatting (1 inch = 1440 DXA).
Pre-bundled build	A library distributed as a single directory with all dependencies and a standalone autoloader, requiring no Composer or package manager.

2. Stakeholders & User Roles

2.1 Overview of Roles

MediRemind supports four distinct user roles, each with specific permissions and responsibilities within the system. Role-based access control is enforced at the session level in PHP.

Role	Access Level	Primary Responsibilities
Patient	Standard	Manage own schedule, view reminders, upload prescriptions, connect/disconnect doctors.
Caretaker	Elevated	Monitor patient schedules, receive missed-dose alerts, update patient schedules, chat with doctors.
Doctor	Verified Professional	Create and update patient prescriptions, manage schedules for linked patients, chat with caretakers.
Admin	Super / System	Verify doctors, manage all users, resolve support tickets, monitor system health.

2.2 Patient

The patient is the primary user of the system. After registering and logging in, the patient can create a personal medicine schedule, mark doses as taken, upload prescription documents, search for and connect with a doctor, and view their full dose history. Patients receive email and SMS reminders before each scheduled dose. When treatment is complete, the patient can disconnect from their doctor, revoking the doctor's access to their schedule.

2.3 Caretaker

A caretaker is linked to one or more patients. Caretakers can view the patient's schedule, update it if the doctor approves, and receive real-time email or SMS alerts when a patient misses a scheduled dose. They can also initiate a chat conversation with the patient's linked doctor for medical consultation without the patient needing to be involved. Caretakers can also upload prescription documents on behalf of the patient.

2.4 Doctor

Doctors must complete a verification process administered by the admin before they can be connected to patients. Once verified, a doctor can appear in patient search results. After a patient connects to a doctor, the doctor receives full read and write access to that patient's medicine schedule. Doctors can create new schedules, update existing ones, and communicate with the patient's caretaker via the built-in chat module. When treatment ends, the patient or doctor can initiate disconnection.

2.5 Administrator

The administrator has a separate login and dashboard that operates independently from the patient-facing application. The admin can view all users across roles, manage accounts, verify doctors, review prescription uploads, monitor missed-dose patterns, respond to support tickets, review flagged chat messages, and monitor the health of the reminder cron job and email/SMS delivery systems.

3. Functional Requirements

3.1 User Authentication & Registration

1. Users must be able to register by providing their name, email, phone number, password, and role (patient, caretaker, or doctor).
2. Passwords must be hashed using PHP's `password_hash()` function with the BCrypt algorithm before storage.
3. Users must log in with their registered email and password. Sessions must be initiated upon successful authentication.
4. Every protected page must verify an active session and the user's role before rendering content.
5. An admin account is created by the system administrator directly in the database and does not use the public registration form.
6. Doctors who register must have their account flagged as 'unverified' until the admin approves them.
7. A password reset flow must be available via a tokenized email link.

3.2 Medicine Schedule Management

8. Patients, caretakers, and doctors can create a medicine schedule for a patient.
9. Each schedule entry must include: medicine name, dosage, unit (mg/ml/tablet), frequency, specific times per day, start date, and optional end date.
10. Multiple medicines can be added to a single patient's schedule.
11. Schedules can be edited or deactivated by the patient, their linked caretaker, or their linked doctor.
12. All changes to schedules must be logged with a timestamp and the ID of the user who made the change.
13. A patient's schedule must be visible on their dashboard grouped by today's date, showing upcoming and past doses.

3.3 Reminder System

14. The system must send a reminder to the patient via email and SMS at least 15 minutes before each scheduled dose time.
15. A cron job running every minute must query all active schedules and identify doses due within the next 15 minutes.
16. If a patient has not marked a dose as taken within 15 minutes after the scheduled time, the system must mark that dose as 'missed'.
17. Upon detecting a missed dose, the system must send an alert email and SMS to the patient's linked caretaker.
18. All reminder and alert events must be logged in the `notification_logs` table with status (sent/failed).

3.4 Prescription Upload & Parsing

19. Patients and caretakers must be able to upload a prescription file in PDF or DOCX format.
20. The system must use smalot/pdfparser (included as a pre-bundled build) for PDF files and PHPOffice/PhpWord (included as a pre-bundled build) for DOCX files to extract text content.
21. Extracted text must be stored in the prescriptions table alongside the original file path.
22. The system must attempt to identify medicine names, dosages, and frequency from the extracted text using pattern matching.
23. Identified data must pre-fill the schedule creation form, which must then be reviewed and confirmed by the patient or doctor before saving.
24. If parsing fails, the file must be flagged for admin review.
25. Uploaded files must be stored in a non-publicly-accessible directory on the server.

3.5 Doctor Connection

26. A patient must be able to search for a verified doctor by name or registration number.
27. The patient sends a connection request, which the doctor must accept before access is granted.
28. Once connected, the doctor has read and write access to that patient's medicine schedule.
29. A patient can only be connected to one doctor at a time.
30. When the patient's condition is resolved, either the patient or the doctor can initiate disconnection.
31. Upon disconnection, the doctor's write access to the patient's schedule must be revoked immediately.
32. The connection history must be preserved in the patient_doctor table with linked_at and unlinked_at timestamps.

3.6 Doctor–Caretaker Chat

33. Caretakers and doctors connected to the same patient must be able to exchange messages through an in-app chat interface.
34. Messages must be stored in the messages table and retrieved using database polling every 3–5 seconds via JavaScript.
35. Each message must be associated with a patient_id so conversations are organized per patient.
36. A new-message email notification must be sent to the recipient if they are not actively viewing the chat.
37. If a user reports a message, it must be flagged for admin review.

3.7 Dose Logging & History

38. Patients must be able to mark a dose as 'taken' from their dashboard.
39. If a dose is not marked taken within the grace window, it must be automatically set to 'missed' by the cron job.
40. Patients and caretakers must be able to view a full dose history, filterable by medicine, date range, and status.
41. Doctors must be able to view the dose history of their connected patients.

3.8 Admin Module

42. The admin must have a dedicated login and dashboard separate from the main application.
43. The admin must be able to view, search, and filter all registered users by role and status.
44. The admin must be able to activate, suspend, or deactivate user accounts.
45. The admin must review and verify or reject doctor registration applications.
46. The admin must have access to a system health page showing cron job last-run time, email delivery stats, and SMS delivery stats.
47. The admin must be able to create, view, and respond to support tickets raised by any user.
48. All admin actions must be logged in the admin_activity_log table.

4. Non-Functional Requirements

4.1 Security

- All passwords must be stored as bcrypt hashes. Plain-text passwords must never be stored or logged.
- Every form submission must include a CSRF token validated on the server side.
- All database queries must use PDO prepared statements to prevent SQL injection.
- Uploaded files must be stored outside the web root and served through a PHP proxy that validates user permissions.
- Admin pages must be protected by a separate session variable (admin_logged_in) and an IP allowlist if deployed on a shared server.
- The admin login must lock out after 5 consecutive failed attempts for 15 minutes.
- HTTPS must be enforced in production using an SSL certificate.

4.2 Performance

- The reminder cron job must complete its full run in under 30 seconds to avoid overlapping with the next minute's execution.
- Database queries on large tables (dose_logs, notification_logs) must use indexed columns (user_id, scheduled_at) for fast lookup.
- Chat polling requests must be lightweight — returning only new messages since the last timestamp — to minimize server load.

4.3 Usability

- The patient dashboard must display today's schedule clearly with a single-click 'Mark as Taken' button for each dose.
- All forms must display inline validation errors without a full page reload where possible.
- The interface must be responsive and usable on mobile browsers without a separate mobile app.

4.4 Reliability

- The cron job health table must be updated on every successful execution so the admin can detect failures.
- All outgoing email and SMS actions must be wrapped in try/catch blocks, with failures logged to notification_logs.
- If a prescription file fails to parse, the system must degrade gracefully — storing the raw file and flagging it for admin review rather than throwing a fatal error.

4.5 Maintainability

- Code must follow an MVC-style folder structure separating models, views, and controllers.
- All database credentials must be stored in a single config.php file outside the web root and never committed to version control.
- Each module must be independently testable by navigating directly to its controller URL with a valid session.

5. Database Design

5.1 Overview

The MediRemind database consists of 13 tables organized around the central users table. All records use auto-increment integer primary keys. Every table that references a user includes a foreign key to users.id. Timestamps use the MySQL DATETIME type and default to CURRENT_TIMESTAMP.

5.2 Table Definitions

5.2.1 users

Stores all registered users across all roles.

Column	Type	Description
id	INT AUTO_INCREMENT PK	Unique user identifier.
name	VARCHAR(150)	Full name of the user.
email	VARCHAR(255) UNIQUE	Login email address.
phone	VARCHAR(20)	Mobile number for SMS notifications.
password_hash	VARCHAR(255)	bcrypt hashed password.
role	ENUM('patient','caretaker','doctor','admin')	User's role in the system.
status	ENUM('active','suspended','deactivated')	Account status, default: active.
is_verified	TINYINT(1)	Doctor verification flag; 1 = verified, 0 = pending.
created_at	DATETIME	Account creation timestamp.
updated_at	DATETIME	Last profile update timestamp.

5.2.2 doctor_profiles

Extended profile for doctor users, linked to users.id.

Column	Type	Description
id	INT AUTO_INCREMENT PK	Record identifier.
doctor_id	INT FK → users.id	Reference to the doctor's user record.
registration_number	VARCHAR(100)	Medical council registration number.
specialization	VARCHAR(150)	Doctor's medical specialization.
hospital	VARCHAR(200)	Current affiliated hospital or clinic.

Column	Type	Description
verified_by	INT FK → users.id	Admin user who verified this doctor.
verified_at	DATETIME	Timestamp of verification.

5.2.3 patient_caretaker

Maps patients to their caretakers (many-to-many relationship).

Column	Type	Description
id	INT AUTO_INCREMENT PK	Record identifier.
patient_id	INT FK → users.id	Patient user reference.
caretaker_id	INT FK → users.id	Caretaker user reference.
linked_at	DATETIME	Timestamp when the relationship was established.

5.2.4 patient_doctor

Tracks connections between patients and doctors, including historical disconnections.

Column	Type	Description
id	INT AUTO_INCREMENT PK	Record identifier.
patient_id	INT FK → users.id	Patient user reference.
doctor_id	INT FK → users.id	Doctor user reference.
status	ENUM('pending','active','disconnected')	Connection status.
linked_at	DATETIME	Timestamp when the connection was accepted.
unlinked_at	DATETIME NULL	Timestamp of disconnection; NULL if still active.

5.2.5 medicines

Master list of medicines referenced by schedules.

Column	Type	Description
id	INT AUTO_INCREMENT PK	Medicine record identifier.
patient_id	INT FK → users.id	The patient this medicine belongs to.
name	VARCHAR(200)	Medicine name (e.g., Metformin).
dosage	VARCHAR(50)	Dosage quantity (e.g., 500).
unit	VARCHAR(20)	Dosage unit: mg, ml, tablet, capsule.

Column	Type	Description
instructions	TEXT	Special instructions (e.g., take with food).
created_by	INT FK → users.id	User who added this medicine.
created_at	DATETIME	Creation timestamp.

5.2.6 schedules

Defines when each medicine should be taken for each patient.

Column	Type	Description
id	INT AUTO_INCREMENT PK	Schedule record identifier.
patient_id	INT FK → users.id	Patient this schedule belongs to.
medicine_id	INT FK → medicines.id	Medicine being scheduled.
times	JSON	Array of time strings e.g. ['08:00','14:00','20:00'].
frequency	VARCHAR(50)	Frequency description e.g. 'Daily', 'Twice daily'.
start_date	DATE	Schedule start date.
end_date	DATE NULL	Schedule end date; NULL for indefinite.
status	ENUM('active','paused','completed')	Current schedule status.
created_by	INT FK → users.id	User who created this schedule.
last_modified_by	INT FK → users.id	User who last modified this schedule.
updated_at	DATETIME	Last modification timestamp.

5.2.7 dose_logs

Records every scheduled dose instance and its outcome.

Column	Type	Description
id	INT AUTO_INCREMENT PK	Log entry identifier.
schedule_id	INT FK → schedules.id	Parent schedule reference.
patient_id	INT FK → users.id	Patient who the dose belongs to.
scheduled_at	DATETIME	Exact date and time the dose was due.
taken_at	DATETIME NULL	Timestamp when patient marked the dose taken.

Column	Type	Description
status	ENUM('pending','taken','missed')	Dose status; updated by patient or cron.

5.2.8 prescriptions

Stores uploaded prescription files and their extracted text content.

Column	Type	Description
id	INT AUTO_INCREMENT PK	Prescription record identifier.
patient_id	INT FK → users.id	Patient the prescription belongs to.
uploaded_by	INT FK → users.id	User who uploaded the file.
file_path	VARCHAR(500)	Server path to the uploaded file (outside web root).
file_type	ENUM('pdf','docx')	Format of the uploaded file.
parsed_text	LONGTEXT	Full extracted text from the prescription file.
parse_status	ENUM('success','failed','pending')	Parsing outcome.
uploaded_at	DATETIME	Upload timestamp.

5.2.9 messages

Stores chat messages between caretakers and doctors for a specific patient.

Column	Type	Description
id	INT AUTO_INCREMENT PK	Message identifier.
patient_id	INT FK → users.id	Patient the conversation is about.
sender_id	INT FK → users.id	User who sent the message.
receiver_id	INT FK → users.id	User who receives the message.
body	TEXT	Message content.
sent_at	DATETIME	Timestamp when the message was sent.
read_at	DATETIME NULL	Timestamp when the receiver opened the message.
is_flagged	TINYINT(1)	Set to 1 if the message is reported for admin review.

5.2.10 notification_logs

Logs every outgoing notification attempt (email or SMS) for auditing.

Column	Type	Description
id	INT AUTO_INCREMENT PK	Log entry identifier.
user_id	INT FK → users.id	Recipient user.
type	ENUM('email','sms')	Notification channel used.
purpose	VARCHAR(100)	Reason: 'reminder', 'missed_dose', 'new_message', etc.
status	ENUM('sent','failed')	Delivery outcome.
error_message	TEXT NULL	Error details if status is 'failed'.
sent_at	DATETIME	Timestamp of the notification attempt.

5.2.11 support_tickets

Column	Type	Description
id	INT AUTO_INCREMENT PK	Ticket identifier.
raised_by	INT FK → users.id	User who submitted the ticket.
subject	VARCHAR(255)	Brief description of the issue.
description	TEXT	Full details of the support request.
status	ENUM('open','in_progress','resolved','closed')	Current ticket status.
priority	ENUM('low','medium','high')	Priority level assigned by admin.
created_at	DATETIME	Ticket creation timestamp.
updated_at	DATETIME	Last update timestamp.

5.2.12 ticket_replies

Column	Type	Description
id	INT AUTO_INCREMENT PK	Reply identifier.
ticket_id	INT FK → support_tickets.id	Parent ticket reference.
replied_by	INT FK → users.id	User (admin or original requester) who replied.
message	TEXT	Reply content.
replied_at	DATETIME	Reply timestamp.

5.2.13 admin_activity_log

Column	Type	Description
id	INT AUTO_INCREMENT PK	Log entry identifier.
admin_id	INT FK → users.id	Admin who performed the action.
action	VARCHAR(255)	Description of the action performed.
target_type	VARCHAR(50)	Type of record affected: user, ticket, prescription, etc.
target_id	INT NULL	ID of the affected record.
ip_address	VARCHAR(45)	Admin's IP address at the time of action.
performed_at	DATETIME	Action timestamp.

5.2.14 cron_health

Column	Type	Description
id	INT AUTO_INCREMENT PK	Record identifier.
job_name	VARCHAR(100)	Name of the cron script.
last_run_at	DATETIME	Timestamp of the most recent successful execution.
status	ENUM('ok','error')	Result of the last run.
notes	TEXT NULL	Error details or run summary.

6. Module Descriptions

6.1 Authentication Module

The authentication module handles registration, login, session management, and password reset for all user roles.

Registration: A single registration form captures name, email, phone, password (confirmed), and role. On submission, the server validates inputs, checks for duplicate emails, hashes the password with `password_hash(PASSWORD_BCRYPT)`, and inserts the record into users. Doctors are inserted with `is_verified = 0`.

Login: The login form accepts email and password. PHP verifies the password against the stored hash using `password_verify()`. On success, `$_SESSION['user_id']` and `$_SESSION['role']` are set and the user is redirected to their role-specific dashboard.

Session Guard: A reusable `auth_check($requiredRole)` function at the top of every protected page checks `$_SESSION['user_id']` exists and `$_SESSION['role']` matches the required role. Failure redirects to `/login`.

Password Reset: The user submits their email. The server generates a random token (`bin2hex(random_bytes(32))`), stores it in `password_resets` table with an expiry of 1 hour, and emails a reset link via PHPMailer. The reset page validates the token and expiry before allowing the password change.

6.2 Schedule Manager Module

This module handles the creation, display, editing, and deactivation of medicine schedules.

Creating a schedule: The form collects medicine name, dosage, unit, instructions, frequency, dose times (added dynamically with JavaScript), start date, and optional end date. On submission, a record is inserted into `medicines` and then into `schedules` with the `times` column stored as a JSON array.

Dashboard view: The patient dashboard queries today's active schedules and joins with `dose_logs` to show whether each dose for today is pending, taken, or missed. Each pending dose shows a 'Mark as Taken' button which triggers a POST request updating the `dose_log` status to 'taken' and setting `taken_at`.

Editing: Caretakers and doctors linked to the patient can access a schedule edit form. All changes are saved with the `last_modified_by` column updated to the editor's user ID.

6.3 Reminder Engine (Cron Module)

The core of the notification system. The file `reminder_cron.php` is executed by the server's cron daemon every minute.

```
# Cron tab entry (server-side configuration)
```

```
* * * * * /usr/bin/php /var/www/mediremind/cron/reminder_cron.php >>
/var/log/mediremind cron.log 2>&1
```

On each execution, the script performs the following steps:

49. Query all dose_logs where status = 'pending' and scheduled_at is between NOW() and NOW() + 15 minutes.
50. For each matching dose, check if a reminder has already been sent (notification_logs). If not, send email and SMS to the patient and log the result.
51. Query all dose_logs where status = 'pending' and scheduled_at < NOW() - 15 minutes (grace window expired).
52. For each expired dose, update status to 'missed' and send an alert to the patient's linked caretaker.
53. Update the cron_health table with the current timestamp and status = 'ok'.
54. If any step throws an exception, log the error to cron_health with status = 'error'.

6.4 Prescription Parser Module

Handles the upload, storage, and automated extraction of prescription information from PDF and DOCX files.

Upload: Files are accepted via a form with enctype='multipart/form-data'. The server validates MIME type (application/pdf or application/vnd.openxmlformats) and file size (max 5MB). Files are renamed to a UUID-based filename and stored in /storage/prescriptions/ outside the web root.

PDF parsing: The project uses smalot/pdfparser without Composer. Download the official pre-bundled build from the releases page on GitHub (e.g. pdfparser-v2.x-bundle.zip). Unzip it into libs/pdfparser/ inside the project. The bundle ships with its own autoloader; include it with require_once 'libs/pdfparser/bootstrap.php'. Then call (new \Smalot\PdfParser\Parser())->parseFile(\$path) to extract all text. The parsed text is stored in prescriptions.parsed_text.

DOCX parsing: PHPOffice/PhpWord is similarly included as a pre-bundled build (download PhpWord-v1.x-bundle.zip from its GitHub releases and unzip into libs/phpword/). Include its autoloader with require_once 'libs/phpword/bootstrap.php'. Then use \PhpOffice\PhpWord\IOFactory::load(\$path) and iterate over all sections, text runs, and table cells to concatenate all readable text.

Library folder layout: Both pre-bundled builds live under libs/ inside the project root, outside the web root. A single require_once block at the top of PrescriptionController.php loads both autoloaders before any parsing calls are made.

```
// In PrescriptionController.php - load both pre-bundled autoloaders
require_once __DIR__ . '/../libs/pdfparser/bootstrap.php';
require_once __DIR__ . '/../libs/phpword/bootstrap.php';

// PDF extraction
$parser = new \Smalot\PdfParser\Parser();
$pdf    = $parser->parseFile($filePath);
$text   = $pdf->getText();
```

```
// DOCX extraction
$word      = \PhpOffice\PhpWord\IOFactory::load($filePath);
$fullText = '';
foreach ($word->getSections() as $section) {
    foreach ($section->getElements() as $element) {
        if (method_exists($element, 'getText')) {
            $fullText .= $element->getText() . ' ';
        }
    }
}
```

Auto-detection: A regex scan of the parsed text looks for patterns such as medicine names (words followed by dosage numbers and units like mg, ml, tablet), timing keywords (morning, afternoon, evening, bedtime), and frequency indicators (once daily, twice daily, OD, BD, TDS). Matches pre-fill the schedule form.

Human review: Pre-filled data is presented to the user in an editable form before saving. No schedule is saved automatically from parsed data.

6.5 Doctor Connection Module

Manages the lifecycle of a patient–doctor relationship from initial connection through disconnection.

Search: Patients search for doctors by name or registration number. The query filters the users table by role = 'doctor' and is_verified = 1, excluding already-connected doctors.

Connection request: Patient submits a connection request. A record is inserted into patient_doctor with status = 'pending'. The doctor receives an email notification.

Acceptance: The doctor's dashboard shows pending connection requests. Clicking Accept updates the record status to 'active' and sets linked_at. The doctor immediately gains access to the patient's schedule.

Disconnection: Either party can initiate disconnection. Clicking Disconnect updates the status to 'disconnected' and sets unlinked_at. The doctor loses write access. All prior schedule changes are retained.

6.6 Chat Module

Provides asynchronous in-app messaging between the caretaker and doctor of a shared patient.

Message polling: The chat interface makes an AJAX GET request to get_messages.php?patient_id=X&after=Y every 4 seconds. The server returns all messages where patient_id = X and sent_at > Y as JSON. JavaScript appends new messages to the chat window.

Sending: Sending a message makes a POST request to send_message.php with the body and patient_id. The server inserts the message into the messages table and triggers a PHPMailer notification if the recipient has not been active in the last 60 seconds.

Flagging: A small flag icon on each message allows a user to report it. Flagging sets `is_flagged = 1` and notifies the admin by email.

6.7 Admin Module

A completely separate application section accessible only to users with role = 'admin'. The admin module is described across the following sub-modules.

6.7.1 Admin Dashboard

The admin landing page displays the following live statistics, each queried from the database on page load: total users by role, active patient–doctor connections, schedules created in the past 7 days, missed doses in the past 24 hours, open support tickets, and pending doctor verifications.

A recent activity feed shows the last 20 entries from `admin_activity_log`. A system health widget shows the last successful cron job run time — highlighted red if more than 2 minutes have elapsed since the last entry.

6.7.2 User Management

A paginated, searchable table lists all users with columns for name, email, role, status, and registration date. From this view, the admin can click any user to open their profile page, which shows linked relationships (doctor, caretakers), login history, and the admin notes field. The admin can change account status (active/suspended/deactivated), trigger a password reset email, and add internal notes.

6.7.3 Doctor Verification

A dedicated page lists all doctors with `is_verified = 0`. Each entry shows the doctor's name, registration number, specialization, hospital, and registration date. The admin clicks Verify to set `is_verified = 1` and log the action in `admin_activity_log`. The doctor is notified by email. Alternatively, the admin can reject and deactivate the account with a reason noted.

6.7.4 Prescription Audit

A table lists all prescription uploads with columns for patient name, uploader, file type, upload date, and parse status. Entries where `parse_status = 'failed'` are highlighted. The admin can download the original file and manually review the parsed text to identify parsing errors. Admin can flag the record as reviewed.

6.7.5 Missed Dose Reports

A date-range filtered view of `dose_logs` where `status = 'missed'`, showing patient name, medicine, scheduled time, and whether an alert was sent. This view helps the admin spot system-wide failures: if many patients missed doses in the same time window, it likely indicates a cron or notification failure. The full table is exportable to CSV using PHP's `fputcsv()` function.

6.7.6 Support Ticket System

Tickets raised by any user appear in a table sortable by status and priority. Clicking a ticket opens a thread view showing the original description and all replies. The admin can reply (which sends the user an email notification), change the status and priority, and close the ticket with a resolution note.

6.7.7 Chat Moderation

A list of all messages where `is_flagged = 1`, showing sender, receiver, patient context, message content, and report timestamp. The admin can review the full thread and take action: warn the user (send an automated email), suspend the account, or dismiss the flag as a false report. All moderation actions are logged.

6.7.8 System Health

A monitoring page with four panels: Cron Job Status (`last_run_at` from `cron_health` — green if within 2 minutes, red if overdue), Email Delivery Rate (ratio of sent to sent+failed in `notification_logs` for the past 24 hours, filtered by `type = 'email'`), SMS Delivery Rate (same query filtered by `type = 'sms'`), and Storage Usage (PHP's `disk_free_space()` and `disk_total_space()` for the uploads directory).

7. Application Page Structure

7.1 Public Pages

Route	File	Description
/	index.php	Landing page with app description and login/register links.
/login	login.php	Login form for all roles.
/register	register.php	Registration form for patient, caretaker, or doctor.
/reset-password	reset_password.php	Password reset request form.
/reset-password?token=X	reset_confirm.php	New password form (requires valid token).

7.2 Patient Pages

Route	Description
/dashboard	Today's schedule with dose status and Mark as Taken buttons.
/schedule/add	Form to add a new medicine and schedule.
/schedule/edit/{id}	Edit an existing schedule entry.
/schedule/history	Full dose history with filters.
/prescriptions	Upload prescription file and view parsed data.
/doctor/search	Search for verified doctors.
/doctor/my-doctor	View current connected doctor and disconnect option.
/tickets/new	Submit a support ticket.
/tickets/my	View all own tickets and admin replies.

7.3 Caretaker Pages

Route	Description
/caretaker/dashboard	Overview of linked patients and their today's dose status.
/caretaker/patient/{id}/schedule	View and edit a specific patient's schedule.
/caretaker/patient/{id}/history	Dose history for a specific patient.
/caretaker/chat/{patient_id}	Chat with the patient's doctor about a specific patient.

7.4 Doctor Pages

Route	Description
/doctor/dashboard	List of connected patients and pending connection requests.
/doctor/patient/{id}/schedule	View, create, and edit schedules for a connected patient.
/doctor/patient/{id}/history	Dose adherence history for a specific patient.
/doctor/chat/{patient_id}	Chat with the patient's caretaker.

7.5 Admin Pages

Route	Description
/admin/login	Separate admin login form.
/admin/dashboard	Stats overview, activity feed, system health widget.
/admin/users	All users table with search and filter.
/admin/users/{id}	Individual user profile with admin actions.
/admin/doctors/verify	List of pending doctor verification requests.
/admin/prescriptions	Prescription upload audit with failed parse flags.
/admin/missed-doses	Missed dose report with CSV export.
/admin/tickets	All support tickets table.
/admin/tickets/{id}	Ticket thread with reply form.
/admin/chat-reports	Flagged chat messages for moderation.
/admin/system-health	Cron status, email/SMS delivery rates, storage.
/admin/activity-log	Full audit trail of all admin actions.

8. Project Folder Structure

The project follows an MVC-inspired organization. All PHP logic is separated from HTML templates. Configuration is kept outside the web root. PHP libraries are included as pre-bundled builds under `libs/` — no Composer or package manager is required.

```

mediremind/
├── config/
│   └── config.php           # DB credentials, SMTP, SMS API keys
├── cron/
│   └── reminder_cron.php    # Scheduled task: reminders + missed dose alerts
├── controllers/
│   ├── AuthController.php
│   ├── ScheduleController.php
│   ├── PrescriptionController.php
│   ├── DoctorController.php
│   ├── ChatController.php
│   ├── TicketController.php
│   └── AdminController.php
├── models/
│   ├── User.php
│   ├── Schedule.php
│   ├── DoseLog.php
│   ├── Prescription.php
│   ├── Message.php
│   └── Ticket.php
├── views/
│   ├── layouts/
│   │   ├── main.php        # Header, nav, footer wrapper
│   │   └── admin.php       # Admin layout wrapper
│   ├── auth/
│   ├── patient/
│   ├── caretaker/
│   ├── doctor/
│   └── admin/
├── helpers/
│   ├── auth_check.php      # Session guard function
│   ├── mailer.php          # PHPMailer wrapper
│   ├── sms.php             # SMS API wrapper (Fast2SMS/Twilio)
│   └── csrf.php            # CSRF token generation and validation
├── libs/
│   ├── phpmailer/          # Pre-bundled PHP libraries (no Composer needed)
│   │   └── bootstrap.php   # PHPMailer – copy src/ from GitHub release
│   ├── pdfparser/         # smalot/pdfparser pre-bundled build
│   │   └── bootstrap.php   # Library autoloader (included in bundle)
│   ├── phpword/           # PHPOffice/PhpWord pre-bundled build
│   │   └── bootstrap.php   # Library autoloader (included in bundle)
│   └── bootstrap.php       # Web root (only this folder is public)
├── public/
│   ├── index.php
│   ├── css/
│   ├── js/
│   └── assets/
├── storage/
│   └── prescriptions/      # Uploaded files (outside web root)
└── .htaccess               # Optional: route all requests through public/

```

8.1 Configuration File

The config.php file stores all environment-specific credentials and must never be committed to version control. Add it to .gitignore immediately.

```
<?php
// config/config.php

define('DB_HOST', 'localhost');
define('DB_NAME', 'mediremind');
define('DB_USER', 'root');
define('DB_PASS', 'your_password');

define('SMTP_HOST', 'smtp.gmail.com');
define('SMTP_USER', 'your@gmail.com');
define('SMTP_PASS', 'your_app_password');
define('SMTP_PORT', 587);

define('SMS_API_KEY', 'your_fast2sms_api_key');
define('SMS_SENDER', 'MEDIRM');

define('STORAGE_PATH', __DIR__ . '/../storage/prescriptions/');
define('APP_URL', 'https://yourdomain.com');
```

9. External Libraries & Dependencies

9.1 PHP Libraries (Manual / Pre-bundled — No Composer Required)

This project does not use Composer. Each PHP library is obtained as a pre-bundled release that includes all its own dependencies and a standalone bootstrap.php autoloader. Download the latest release archive from each library's GitHub releases page, unzip it into the corresponding libs/ subdirectory listed below, and require the bootstrap.php file at the top of the relevant controller.

Library	libs/ subdirectory	Download source	Purpose
PHPMailer	libs/phpmailer/	github.com/PHPMailer/PHPMailer — copy src/ folder from release zip	Sending SMTP emails for reminders, alerts, and ticket replies.
smalot/pdfparser	libs/pdfparser/	GitHub releases — download pre-bundled pdfparser-vX.x-bundle.zip; includes bootstrap.php	Extracting text content from uploaded PDF prescription files.
PHPOffice/PhpWord	libs/phpword/	GitHub releases — download PhpWord-vX.x-bundle.zip; includes bootstrap.php	Extracting text content from uploaded DOCX prescription files.

PHPMailer does not ship a bundle zip, but it has no external dependencies. Simply copy the src/ directory from the release into libs/phpmailer/src/ and require each class file manually, or use the provided autoloader pattern shown in the PHPMailer README.

9.2 Third-Party API Services

Service	Use Case	Integration Method
Fast2SMS	Sending SMS reminders and alerts within India.	REST API call using PHP file_get_contents() or cURL.
Twilio	International SMS alternative to Fast2SMS.	Official PHP SDK or REST API with HTTP Basic Auth.
Gmail SMTP	Sending emails via PHPMailer.	App Password with 2FA enabled on a Gmail account.

9.3 Front-End Libraries (CDN)

Library	Version	Purpose
Bootstrap	5.3	Responsive layout, forms, buttons, and modals.
jQuery	3.7	Simplified DOM manipulation and AJAX for chat polling.
Flatpickr	4.6	Date and time picker for schedule creation forms.
Font Awesome	6.x	Icons for dashboard status indicators.

10. Security Considerations

10.1 Input Validation & SQL Injection Prevention

All database interactions must use PDO with prepared statements. No query must ever concatenate user input directly into an SQL string. All user inputs from forms must be validated server-side using PHP filter functions and custom validation before use.

```
// Correct: Prepared statement with PDO
$stmt = $pdo->prepare('SELECT * FROM users WHERE email = :email');
$stmt->execute([':email' => $email]);
$user = $stmt->fetch(PDO::FETCH_ASSOC);

// Wrong: Never do this
// $result = $pdo->query("SELECT * FROM users WHERE email = '" . $email . "'");
```

10.2 CSRF Protection

Every HTML form must include a hidden CSRF token field. The token is generated on page load, stored in the session, and validated on form submission. Any POST request without a valid matching token must be rejected with a 403 response.

```
// helpers/csrf.php
function generate_csrf_token(): string {
    if (empty($_SESSION['csrf_token'])) {
        $_SESSION['csrf_token'] = bin2hex(random_bytes(32));
    }
    return $_SESSION['csrf_token'];
}

function validate_csrf_token(string $token): bool {
    return hash_equals($_SESSION['csrf_token'] ?? '', $token);
}

// In every form:
// <input type='hidden' name='csrf token' value='<?= generate csrf token() ?>'>
```

10.3 File Upload Security

- Only PDF and DOCX files are accepted. MIME type is validated using `finfo_file()`, not the browser-provided MIME type.
- Files are renamed to a UUID-based name to prevent directory traversal and filename-based attacks.
- Uploaded files are stored outside the public web root in `/storage/prescriptions/`. Access is controlled by a PHP file-serving script that validates user permissions before streaming the file.
- The upload directory must not have PHP execution permissions.

10.4 Admin Security

- Admin session uses a separate key: `$_SESSION['admin_logged_in'] = true`, distinct from patient sessions.
- Admin login tracks failed attempts in the session. After 5 failures, a 15-minute lockout is enforced.
- All admin write actions (status changes, verifications, ticket replies) use POST with CSRF protection.
- Admin activity log records the IP address of every significant action for forensic auditing.

11. Implementation Plan

11.1 Development Phases

Phase	Tasks	Estimated Duration
Phase 1 — Setup	Project folder structure, config.php, database schema SQL, download and place pre-bundled library files under libs/, Bootstrap template.	2 days
Phase 2 — Auth	Registration, login, session management, role-based guards, password reset.	3 days
Phase 3 — Schedule	Medicine CRUD, schedule creation form, patient dashboard with today's doses, Mark as Taken.	4 days
Phase 4 — Reminders	Cron job script, PHPMailer integration, Fast2SMS integration, notification_logs.	3 days
Phase 5 — Prescriptions	File upload, PDF parser (pdfparser pre-bundle), DOCX parser (PhpWord pre-bundle), pre-fill form, admin flagging.	3 days
Phase 6 — Doctor Connect	Search, connection request, acceptance flow, disconnection, access control.	3 days
Phase 7 — Chat	Messages table, polling AJAX, send message, flag message, email notification.	3 days
Phase 8 — Admin	Admin login, dashboard stats, user management, doctor verification, tickets, health page.	5 days
Phase 9 — Testing	Manual testing of all flows, edge cases, security checks, cron simulation.	3 days
Phase 10 — Deployment	Deploy to shared hosting or VPS, configure cron, HTTPS setup, final review.	2 days

11.2 Recommended Development Environment

- Local server: XAMPP or Laragon (Windows) / MAMP (Mac) / standalone PHP + MySQL on Linux.
- PHP version: 8.1 or higher for named arguments and enum support.
- MySQL version: 8.0+ for native JSON column support.
- Code editor: VS Code with PHP Intelephense extension.
- Version control: Git with .gitignore covering config.php and storage/. The libs/ directory should be committed so the pre-bundled library files are part of the repository.

11.3 Testing Checklist

- Registration and login for all four roles.

- Session guard: directly access a protected URL without logging in — must redirect to login.
- Schedule creation with multiple dose times — verify correct entries in `dose_logs`.
- Cron job simulation: run `reminder_cron.php` manually and verify emails/SMS are sent and logged.
- Manually set a dose's `scheduled_at` to a past time and run the cron — verify it is marked as 'missed' and the caretaker is alerted.
- Upload a PDF prescription and verify text is extracted and the schedule form is pre-filled.
- Doctor connection flow: request, accept, edit schedule as doctor, disconnect, verify access is revoked.
- Chat: send a message, verify it appears for the recipient within 4 seconds without a page refresh.
- Admin: verify a doctor, suspend a user, reply to a ticket — verify all are logged in `admin_activity_log`.
- CSRF: remove the token from a form POST and verify the server rejects it with a 403 error.

12. Future Enhancements

The following features are out of scope for the current mini project but are recommended for future development:

- Progressive Web App (PWA) support for offline access and push notifications via the browser.
- WebSocket-based real-time chat to replace the polling mechanism for lower latency.
- AI-assisted prescription parsing using a language model API to improve accuracy beyond regex matching.
- Multi-language support (i18n) for regional language interfaces.
- Telemedicine video call integration between patients, caretakers, and doctors using WebRTC.
- Family dashboard allowing a single caretaker to manage multiple patients from one view.
- Medicine inventory tracking with low-stock alerts reminding patients to refill prescriptions.
- Analytics dashboard for doctors showing patient-level adherence rates across their patient list.
- Integration with national health record systems or HL7 FHIR standards for interoperability.

13. Appendix

13.1 Database Schema (SQL)

The following SQL can be executed directly in MySQL to create the complete MediRemind database schema.

```
CREATE DATABASE IF NOT EXISTS mediremind CHARACTER SET utf8mb4 COLLATE
utf8mb4_unicode_ci;
USE mediremind;

CREATE TABLE users (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(150) NOT NULL,
  email VARCHAR(255) UNIQUE NOT NULL,
  phone VARCHAR(20),
  password_hash VARCHAR(255) NOT NULL,
  role ENUM('patient','caretaker','doctor','admin') NOT NULL,
  status ENUM('active','suspended','deactivated') DEFAULT 'active',
  is_verified TINYINT(1) DEFAULT 1,
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
  updated_at DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);

CREATE TABLE schedules (
  id INT AUTO_INCREMENT PRIMARY KEY,
  patient_id INT NOT NULL,
  medicine_id INT NOT NULL,
  times JSON NOT NULL,
  frequency VARCHAR(50),
  start_date DATE NOT NULL,
  end_date DATE,
  status ENUM('active','paused','completed') DEFAULT 'active',
  created_by INT NOT NULL,
  last_modified_by INT,
  updated_at DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  FOREIGN KEY (patient_id) REFERENCES users(id),
  FOREIGN KEY (created_by) REFERENCES users(id)
);

CREATE TABLE dose_logs (
  id INT AUTO_INCREMENT PRIMARY KEY,
  schedule_id INT NOT NULL,
  patient_id INT NOT NULL,
  scheduled_at DATETIME NOT NULL,
  taken_at DATETIME,
  status ENUM('pending','taken','missed') DEFAULT 'pending',
  INDEX idx_patient_scheduled (patient_id, scheduled_at),
  FOREIGN KEY (schedule_id) REFERENCES schedules(id),
  FOREIGN KEY (patient_id) REFERENCES users(id)
);
```

13.2 Library Setup Reference

Since Composer is not used, the steps below summarise how to get each PHP library into place.

```
# 1. PHPMailer – no bundle needed, no external dependencies
#   Download the release zip from github.com/PHPMailer/PHPMailer/releases
#   Copy the src/ folder into libs/phpmailer/src/
#   In mailer.php:
use PHPMailer\PHPMailer\PHPMailer;
require_once __DIR__ . '/../libs/phpmailer/src/Exception.php';
require_once __DIR__ . '/../libs/phpmailer/src/PHPMailer.php';
require_once __DIR__ . '/../libs/phpmailer/src/SMTP.php';

# 2. smalot/pdfparser – pre-bundled build
#   Download pdfparser-vX.x-bundle.zip from the GitHub releases page
#   Unzip into libs/pdfparser/ (the zip already contains bootstrap.php)
require_once __DIR__ . '/../libs/pdfparser/bootstrap.php';

# 3. PHPOffice/PhpWord – pre-bundled build
#   Download PhpWord-vX.x-bundle.zip from the GitHub releases page
#   Unzip into libs/phpword/ (the zip already contains bootstrap.php)
require_once __DIR__ . '/../libs/phpword/bootstrap.php';
```

13.3 References

- PHP Manual: <https://www.php.net/manual/en/>
- MySQL 8.0 Reference: <https://dev.mysql.com/doc/refman/8.0/en/>
- PHPMailer GitHub: <https://github.com/PHPMailer/PHPMailer>
- smalot/pdfparser GitHub Releases: <https://github.com/smalot/pdfparser/releases>
- PHPOffice/PhpWord GitHub Releases: <https://github.com/PHPOffice/PhpWord/releases>
- Fast2SMS API Docs: <https://docs.fast2sms.com/>
- Twilio PHP SDK: <https://www.twilio.com/docs/libraries/php>
- OWASP PHP Security Cheat Sheet: <https://cheatsheetseries.owasp.org/>